

Contents

Rapport de projet — Prédiction du prix des locations au Sénégal	1
1. Provenance et caractéristiques des données	1
1.1 Origine	1
1.2 Exploration et qualité des données	1
1.3 Construction des features	2
2. Choix de modélisation et comparaison	3
2.1 Modèles comparés	3
2.2 Résultats	3
2.3 Tracking et registre de modèles	4
3. Choix d'architecture	4
4. Limites connues	5
Annexe — Reproduire les résultats de ce rapport	5

Rapport de projet — Prédiction du prix des locations au Sénégal

Projet M2 DSIA — MLOps Auteurs : Rokhaya Dieng, Ousmane Faye **Dépôt :** <https://github.com/ousmanegithub/Senegal-Rental-Price> **Date :** 4 août 2026

1. Provenance et caractéristiques des données

1.1 Origine

Le jeu de données a été constitué par **scraping** d'annonces de location immobilière au Sénégal sur la plateforme **NeoBien** (neobien.com), un site d'annonces immobilières sénégalais, à usage strictement académique dans le cadre de ce projet. La collecte n'entre pas dans le périmètre du code de production : elle a été menée en amont et n'est ni versionnée ni ré-exécutée par le pipeline (conformément au sujet, `data/raw/` est exclu du dépôt via `.gitignore`).

Le fichier brut obtenu, `data/raw/locations.csv` (CSV, séparateur `;`, encodage UTF-8 avec BOM), contient **166 annonces** décrites par 14 colonnes : identifiant, ville, quartier, type de bien, surface, indicateur « surface estimée » (calculée par le scraper via une expression régulière sur le titre lorsque la surface n'était pas explicitement renseignée), nombre de pièces, nombre de chambres, statut meublé, liste d'équipements, loyer mensuel (variable cible), titre de l'annonce, adresse et date de publication.

1.2 Exploration et qualité des données

La phase d'exploration est tracée intégralement dans `notebooks/01_exploration.ipynb`, séparée du code de production comme l'exige le sujet. Elle a mis en évidence cinq problèmes de qualité, chacun traité par une règle explicite et testée dans `src/senegal_rental_price/data/preprocessing.py` :

#	Problème observé	Traitement retenu
1	La colonne meuble est très peu renseignée (3 biens marqués meublés) alors que le token meuble apparaît 60 fois dans la colonne equipements	Réconciliation : un bien est considéré meublé si la colonne <i>ou</i> les équipements l'indiquent → 53 biens meublés au lieu de 3
2	Plusieurs annonces classées « maison » sont en réalité des locaux commerciaux, bureaux ou entrepôts (loyers de 3 500 à 12 000 FCFA, très en dehors de la distribution résidentielle), plus un « immeuble R+1 » à 15 000 000 FCFA	Filtrage par mots-clés du titre (commercial, bureau, entrep, local, immeuble) combiné à des bornes de plausibilité prix/surface (conf/data/default.yaml)
3	Une seule annonce à « Diakhirate », hors du périmètre géographique visé (Dakar / région de Thiès)	Écartée — restriction aux villes connues (Dakar, Thiès)
4	Valeurs manquantes sur nb_pieces (6 lignes) et nb_chambres (7 lignes)	Imputation par la médiane par type de bien , avec repli sur la médiane globale si un type entier est concerné
5	Colonne adresse vide dans 163 lignes sur 166	Colonne supprimée (non exploitable)

Après nettoyage, le jeu passe de **166 à 155 annonces** (11 écartées : biens commerciaux et valeurs aberrantes), confirmé par l'exécution réelle du pipeline :

Données brutes chargées : 166 lignes, 14 colonnes

Filtrage : 166 -> 155 lignes (11 écartées : commercial/aberrant)

Nettoyage terminé : 155 lignes propres

La distribution géographique reste conforme aux données brutes : 94 annonces à Thiès (majoritairement la Petite-Côte : Saly, Ngaparou, Somone, Nguérigne), 71 à Dakar, réparties entre appartements (53) et maisons/villas (18 à Dakar, 80 à Thiès).

1.3 Construction des features

src/senegal_rental_price/features/build_features.py transforme les données nettoyées en une matrice de 19 features numériques : 3 variables continues (surface, nb_pieces, nb_chambres), des indicateurs binaires (meublé, surface estimée), le nombre d'équipements, un indicateur de « quartier premium » (présence d'un

des 9 quartiers prisés — Almadies, Ngor, Point E, Mermoz, Fann, Plateau, Saly, Ngaparou, Somone — recherché par sous-chaîne insensible à la casse), 8 indicateurs d'équipement (piscine, climatisation, gardiennage, parking, jardin, terrasse, salle de sport, wifi) et l'encodage one-hot de la ville et du type de bien.

Un point de conception important : le **vocabulaire catégoriel est figé** dans une configuration (FeatureConfig) plutôt que déduit dynamiquement des données présentes (ce qu'aurait fait un `pd.get_dummies` naïf). Cela garantit que l'entraînement (plusieurs centaines de lignes) et l'inférence (un seul bien reçu par l'API) produisent **exactement le même schéma de colonnes**, condition nécessaire au bon fonctionnement du modèle sérialisé.

2. Choix de modélisation et comparaison

2.1 Modèles comparés

Conformément au sujet, trois modèles ont été entraînés et comparés via Hydra (`conf/model/{ridge,random_forest,xgboost}.yaml`) et trackés avec MLflow :

- **Ridge** (baseline linéaire régularisée) ;
- **Random Forest** (modèle d'ensemble, 200 arbres, profondeur max 15) ;
- **XGBoost** (gradient boosting).

La comparaison est reproductible en une seule commande grâce au multirun Hydra :

```
python -m senegal_rental_price.models.train -m model=ridge,random_forest,xgboost
```

2.2 Résultats

Split train/test 80/20 (124 exemples d'entraînement, 31 de test), graine fixe (`seed=42`). Résultats obtenus lors de l'exécution de référence :

Modèle	RMSE (FCFA)	MAE (FCFA)	R ²
Ridge	516 651	368 349	0,327
Random Forest	495 603	380 791	0,381
XGBoost	515 463	400 632	0,330

À insérer ici : capture d'écran de la vue de comparaison des runs dans l'interface MLflow (`docker compose -f docker/docker-compose.yml up -d mlflow`, puis `http://localhost:5000`, expérience `senegal-rental-price`), illustrant les trois runs et leurs paramètres/métriques.

Random Forest obtient le meilleur compromis RMSE/R² et a été retenu comme modèle servi par l'API. L'écart entre les trois modèles reste modeste, ce qui est cohérent avec la taille réduite du jeu d'entraînement (124 exemples) : au lieu d'un des deux plus visibles.

2.3 Tracking et registre de modèles

Chaque run est enregistré dans MLflow (paramètres d'entraînement, métriques, artefact du modèle au format cloudpickle), et le modèle est également publié dans le **Model Registry MLflow** sous le nom `senegal-rental-price-model`. En production, l'API peut charger l'estimateur directement depuis le registre (variable d'environnement `API_MLFLOW_MODEL_URI`, ex. `models:/senegal-rental-price-model/Production`) avec repli automatique sur l'artefact local (`models/model.joblib`) si le registre est indisponible — ce qui garantit que le service reste démarrable hors ligne (démonstration, CI, conteneur isolé).

En parallèle, chaque entraînement sérialise localement un « bundle » complet (estimateur + vocabulaire de features figé + métadonnées) via `joblib`, ce qui découple totalement l'API de la logique d'entraînement : elle n'a besoin que de ce fichier pour prédire.

3. Choix d'architecture

Le projet suit la structure imposée par le sujet (`conf/`, `data/`, `src/`, `api/`, `frontend/`, `tests/`, `docker/`) sans écart significatif. Quelques précisions et choix documentés :

- **Séparation stricte notebook / production** : `notebooks/01_exploration.ipynb` ne contient aucune logique réutilisée telle quelle ; chaque décision qui y est prise (bornes de filtrage, réconciliation meuble, imputation) est ensuite implémentée et testée indépendamment dans `src/senegal_rental_price/data/preprocessing.py`.
- **ModelBundle** (`src/senegal_rental_price/models/predict.py`) regroupe l'estimateur, la configuration de features et les métadonnées en un seul objet sérialisé, pour que l'API n'ait à charger qu'un seul artefact au démarrage (jamais par requête, via le gestionnaire `lifespan` de FastAPI).
- **Configuration Hydra multi-fichiers** (`conf/model/*.yaml`, `conf/data/default.yaml`) : aucun hyperparamètre, chemin ou seuil de nettoyage n'est codé en dur ; tout est surchargeable en ligne de commande (`model=xgboost model.params.max_depth=8`).
- **API/Frontend découplés** : le frontend Streamlit n'exécute aucune logique métier, il ne fait qu'appeler l'API HTTP — l'API reste la seule source de vérité pour la prédiction, la validation et les règles de cohérence (ex. `nb_chambres ≤ nb_pieces`).
- **Docker multi-stage + utilisateur non-root** pour l'API, le frontend et un serveur MLflow additionnel (`docker/Dockerfile.mlflow`, non exigé par le sujet mais ajouté pour permettre une démonstration complète du tracking sans dépendance à un service MLflow externe). Les modèles entraînés ne sont **jamais embarqués dans l'image** : `models/` est monté en volume lecture seule par `docker-compose.yml`, ce qui sépare le cycle de vie du code (image versionnée) de celui des artefacts de modèle (mis à jour par ré-entraînement, sans reconstruire l'image).
- **CI GitHub Actions** en deux jobs (`quality` puis `docker-build`, avec `needs: quality`) : le job Docker ne se déclenche que si le lint, le typage et les tests (couverture $\geq 70\%$) sont au vert.

4. Limites connues

- **Taille du jeu de données** : 155 annonces après nettoyage est un volume restreint pour un modèle de régression ; les métriques ($R^2 \approx 0,33-0,38$) reflètent cette contrainte plus qu'une limite de méthode. Le sujet précise explicitement que la performance prédictive n'est pas le critère d'évaluation principal.
- **Couverture géographique** : seules deux zones (Dakar et la région de Thiès / Petite-Côte) sont couvertes ; le modèle ne généralise pas à Saint-Louis, Mbour (hors Petite-Côte) ou d'autres villes malgré leur mention dans l'énoncé, faute de données collectées pour ces zones.
- **Encodage des équipements et quartiers premium** : la liste de quartiers « premium » et d'équipements reconnus est figée manuellement à partir de l'exploration ; elle ne s'auto-ajuste pas si de nouveaux quartiers ou équipements apparaissent dans des données futures (nécessiterait un réentraînement avec une configuration mise à jour).
- **Absence de test d'intrusion / charge** sur l'API : la robustesse fonctionnelle (validation Pydantic, codes d'erreur) est testée, mais aucun test de performance sous charge n'a été réalisé.
- **Persistance MLflow** : le serveur MLflow conteneurisé utilise un backend SQLite local monté en volume (mlflow-data/) — adapté à une démonstration, mais pas à un déploiement multi-utilisateur en production (nécessiterait une base PostgreSQL/MySQL partagée).
- **Pas de ré-entraînement automatisé** : le passage d'un nouveau modèle en production (mise à jour de models/model.joblib ou promotion d'une version dans le Model Registry) reste une opération manuelle ; aucun pipeline de ré-entraînement périodique n'a été mis en place.

Annexe — Reproduire les résultats de ce rapport

```
pip install -e ".[dev,frontend]"
# Placer data/raw/locations.csv (cf. data/README.md)
python -m senegal_rental_price.models.train -m model=ridge,random_forest,xgboost
docker compose -f docker/docker-compose.yml up -d mlflow # UI sur http://localhost:50
python -m senegal_rental_price.models.train model=random_forest # fige l'artefact ser
uvicorn api.main:app --reload
streamlit run frontend/app.py
pytest --cov=src --cov-fail-under=70
```